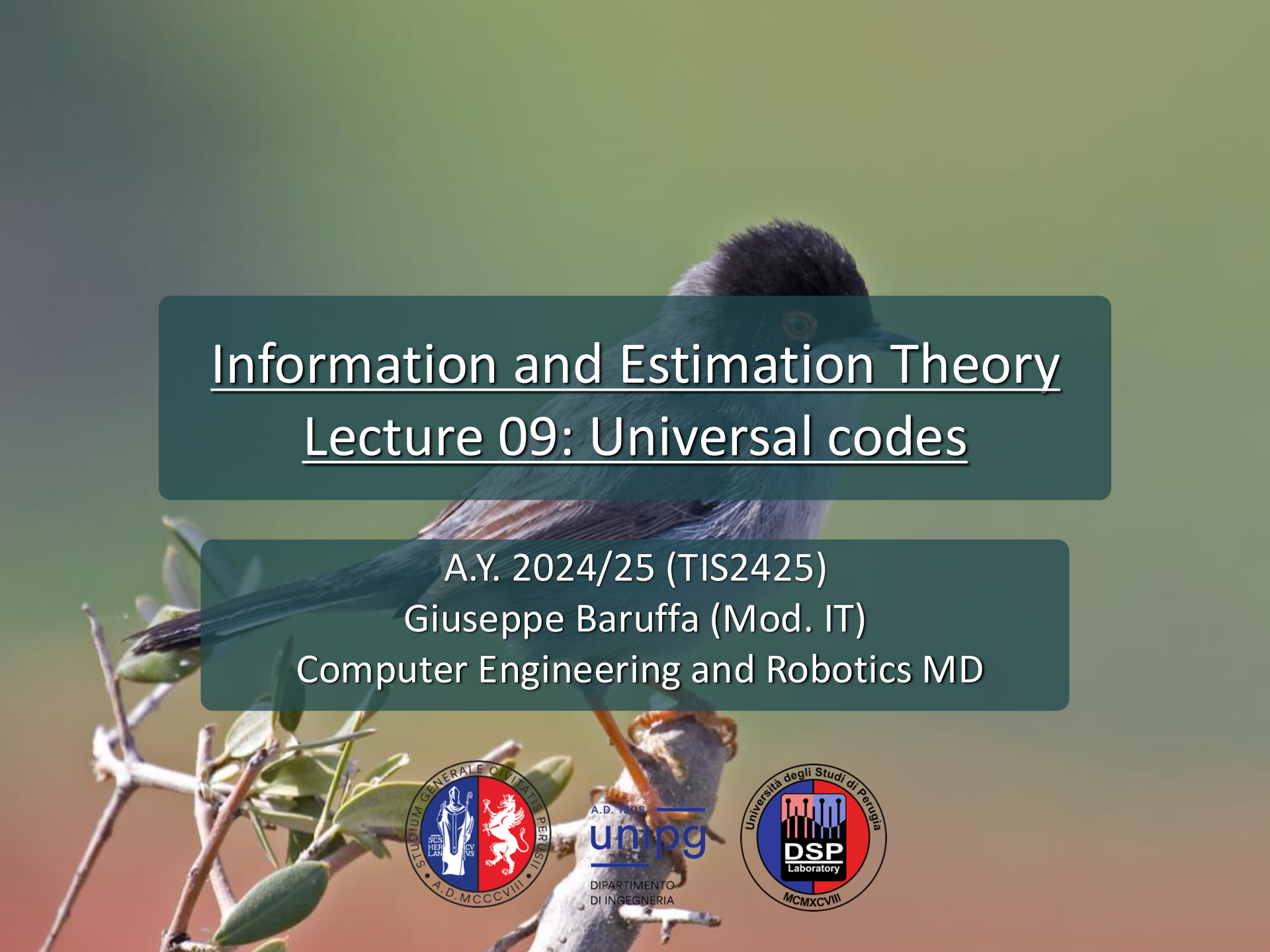


Information and Estimation Theory
Lecture 09: Universal codes

A.Y. 2024/25 (TIS2425)

Giuseppe Baruffa (Mod. IT)

Computer Engineering and Robotics MD

DIPARTIMENTO DI INGEGNERIA

Idea of universal source coding

- We introduced the problem of finding the shortest representation of a source
  - We showed that the entropy is the fundamental lower limit on the expected length of any uniquely decodable representation
- We also showed that if we know the PMF for the source, we can use the Huffman algorithm to construct the optimal (minimal expected length) code for that PMF

- For many practical situations, however, the probability distribution underlying the source may be unknown, and we cannot apply these methods directly
- Instead, all we know is a class of distributions

Idea of universal source coding

- One possible approach (with issues) is to
  - wait until we have all the data (introduces latency)
  - estimate the PMF, e.g., with the histogram (needs a lot of samples)
  - use the estimated PMF to construct the best code (divergence wrt the best)
  - compress the data using this code (needs all the previous steps)
- This 2-pass (*offline*) procedure is used in some applications where there is a small amount of data to be compressed

- But there are situations in which it is not feasible to make two passes over the data, and it is desirable to have a 1-pass (or *online*) algorithm to compress the data that "learns" the PMF and uses it to compress the incoming symbols
- We show the existence of such algorithms that do well for any distribution within a class of distributions

Idea of universal source coding

- In other cases, there is no PMF underlying the data
  - We only have a sequence of outcomes, e.g., text, music, pictures
- How well can we compress the sequence?
- If we do not put any restrictions on the class of algorithms, we get a meaningless answer
  - There is always a function that compresses a particular sequence to 1 bit while leaving every other sequence uncompressed
    - This function is clearly "overfitted" to the data
- However, if we compare our performance to that achievable by optimal word assignments with respect to Bernoulli PMFs or *k*th-order Markov SPs, we obtain more interesting answers

- The ultimate answer for compressibility for an individual sequence is the *Kolmogorov complexity* of the sequence

Idea of universal source coding

- We will describe Golomb-rice coding, also known as run-length coding,
- We will describe *arithmetic coding*, an extension of the Shannon–Fano–Elias's code, which permits incremental encoding and decoding of sequences of source symbols
- We then describe two basic versions of the class of adaptive dictionary compression algorithms called *Lempel–Ziv*
  - Asymptotic optimality for these algorithms: in the limit, they achieve the entropy rate for any *stationary ergodic source*

- **Ergodic process:** its ensemble averages equal the sample averages, e.g.,

Mean-ergodicity

$$E\{X\} = \lim_{N\to\infty} \frac{1}{N} \sum_{i=1}^{N} x_i$$

Golomb-Rice codes

- The Golomb-Rice family of codes is designed to encode integers such that the larger *x*, the lower its probability *p*(*x*)
- The simplest GR code is the **unary code**: for *x*>0 it is simply *x* '1's followed by a '0'
  - The code for *x*=4 is '11110', and the code for *x*=7 is '11111110'
- The unary code is the same as the Huffman code for the semi-infinite alphabet $\mathcal{X}$={1,2,3,...} for $\mathrm{X} \sim p(x) = 2^{-x}$
  - The Huffman code is optimal (dyadic), then the unary code is also optimal
- Improved coding schemes split the integer into two parts
  - Represent one part with a unary code and the other with a different code
- Variations of these codes are used in video coding (H.264) and satellite communications (CCSDS)

Golomb code

- Family of codes parameterized by $M>0$
- It represents an integer $x \geq 0$ using

$$q = \left\lfloor \frac{x}{M} \right\rfloor \qquad r = x - qM = x \bmod M$$

  - q is the quotient and r is the remainder
  - Optimal for $p(x) = p^x(1-p)$ when $M = \left\lceil -\frac{1}{\log p} \right\rceil$
- q=0,1,2,… is unary encoded
- r=0,1,2,…,M−1 uses $\lceil \log(M) \rceil$ bits
- Or, better
  - Uses $\lfloor \log(M) \rfloor$ bits for the first $2^{\lceil \log(M) \rceil} - M$ values of r, and $\lceil \log(M) \rceil$ bits to represent $r' = r + 2^{\lceil \log(M) \rceil} - M$ for the rest
  - Extension to $x \in \mathbb{Z}$: represent negative numbers with $2|x| - 1$ and positive numbers with $2|x|$, then encode

M=5

x	q	r	r'	C(x)
0	0	0	-	000
1	0	1	-	001
2	0	2	-	010
3	0	3	6	0110
4	0	4	7	0111
5	1	0	-	1000
6	1	1	-	1001
7	1	2	-	1010
8	1	3	6	10110
9	1	4	7	10111
10	2	0	-	11000
11	2	1	-	11001
12	2	2	-	11010
13	2	3	6	110110
14	2	4	7	110111
15	3	0	-	111000
…	…	…	…	…

Arithmetic coding

- Huffman coding is optimal for encoding a RV with a known PMF that must be encoded symbol by symbol
- However, due to the integral CW length restriction, there could be a loss of up to 1 bit per symbol in coding efficiency
- We could alleviate this loss by using blocks of input symbols
  - The complexity of this approach increases exponentially with block length

- In **arithmetic coding**, instead of a sequence of bits, we represent a symbol by a subinterval of the unit interval
- The code for a sequence of symbols is an interval whose length decreases as we add more symbols to the sequence
- This coding scheme is incremental (the code for an extension can be calculated simply from the code for the original sequence) and has CW lengths not restricted to be integral

Arithmetic coding

- Motivation is based on Shannon-Fano-Elias codes

- **Lemma:** Let $Y \sim F(y)$ be a continuous RV and F its CDF. Let $U = F(Y)$ (i.e., U is a function of Y defined by its CDF).
  Then U is uniformly distributed on [0,1], $U \sim \mathcal{U}[0,1]$

- *Proof:* since $F(y) \in [0,1]$, the range of U is [0, 1]. Also, for $u \in [0,1]$,

$$F_U(u) = \Pr\{U \leq u\} = \Pr\{F(Y) \leq u\} = \Pr\{Y \leq F^{-1}(u)\} = F\left(F^{-1}(u)\right) = u$$

$$f_U(u) = \frac{d}{du} F_U(u) = 1$$

Arithmetic coding

- Take an infinite seq. of RVs $X_1, X_2, \ldots$ from finite alph. $\mathcal{X}=0,1,2,\ldots,m$
- For any sequence $x_1, x_2, \ldots$, we can place **0.** at the front and consider it as a real number (base $m+1$) between 0 and 1
- Let X be the real-valued RV $X = 0.X_1X_2\cdots$, then X has the CDF

Note the ≤

$$F_X(x) = \Pr\{X \le x = 0.x_1x_2\cdots\} = \Pr\{0.X_1X_2\cdots \le 0.x_1x_2\cdots\}$$
$$= \Pr\{X_1 < x_1\} + \Pr\{X_1 = x_1, X_2 < x_2\} + \cdots$$

Note the < ; $0.x_1\ldots$; Note the <

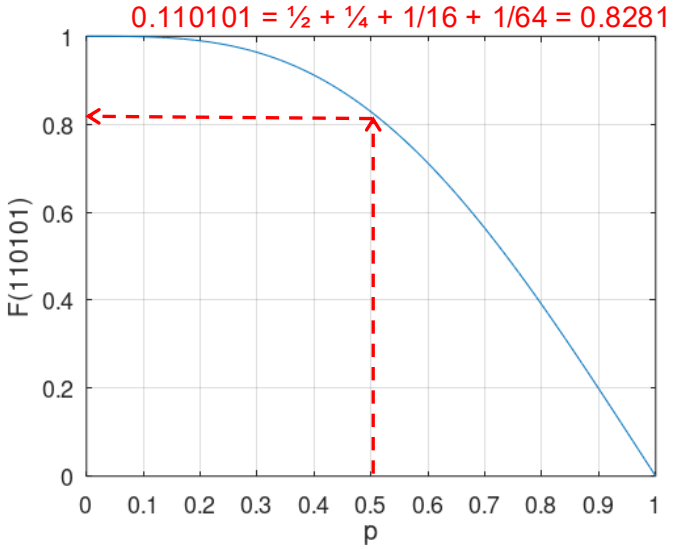
- Let $U = F_X(X) = F_X(0.X_1X_2\cdots) = 0.F_1F_2\cdots$ Binary digits
- If the CDF on infinite sequences X^∞ has no *atoms* (i.e., it is practically continuous), then, by the previous lemma, $U \sim \mathcal{U}[0, 1]$ and the bits $F_1F_2\cdots$ are i.i.d. and $F_i \sim$ Bern(1/2) Equiprobable 1s and 0s
  - They are independent and uniformly distributed on the set {0, 1}

Remember that entropy in this case is 1 bit

- These bits are therefore **incompressible** and already form a compressed representation of the sequence $0.X_1X_2\cdots$

Example

- Let $X_1, X_2, \ldots, X_n \sim \text{Bern}(p)$ $\mathcal{X} = \{0, 1\}$ with PMF $\{q, p\} = \{1\text{-}p, p\}$
- Then the sequence $x^6 = 110101$ maps into



X_1 must be 0, so this prob. is q — X_1 is 1 with prob. p

$$F(x^n) = \Pr\{X_1 < 1\} + \Pr\{X_1 = 1, X_2 < 1\}$$

This is clearly impossible

$$+ \Pr\{X_1 = 1, X_2 = 1, X_3 < 0\}$$
$$+ \Pr\{X_1 = 1, X_2 = 1, X_3 = 0, X_4 < 1\}$$
$$+ \Pr\{X_1 = 1, X_2 = 1, X_3 = 0, X_4 = 1, X_5 < 0\}$$
$$+ \Pr\{X_1 = 1, X_2 = 1, X_3 = 0, X_4 = 1, X_5 = 0, X_6 < 1\}$$
$$= q + pq + p^2 \cdot 0 + p^2 q \cdot q + p^2 qp \cdot 0 + p^2 qpqq$$
$$= q + pq + p^2 q^2 + p^3 q^3$$
$$= (1-p)\left(1 + p\left(1 + p(1-p)\left(1 + p(1-p)\right)\right)\right)$$

where each term is easily computed from the previous terms

Arithmetic coding

- In general, for an arbitrary <u>binary</u> process $\{X_i\}$

$$F(x^n) = \sum_{k=1}^{n} p\left(x^{k-1}0\right)x_k$$

F(110101) = p(0)·1 + p(10)·1 + p(110)·0
+ p(1100)·1 + p(11010)·0
+ p(110100)·1

- The probability transform thus forms an invertible mapping from infinite source sequences to incompressible infinite binary sequences
- What happens on finite sequences?
- Let $X_1, X_2, \ldots, X_n$ be a sequence of binary RVs of length n, and let $x_1, x_2, \ldots, x_n$ be a particular outcome
- Treat this sequence as representing an interval $[0.x_1x_2\cdots x_n000\cdots, 0.x_1x_2\cdots x_n1111\cdots)$, or equivalently, $[0.x_1x_2\cdots x_n, 0.x_1x_2\cdots x_n + (1/2)^n)$
- This is the set of infinite sequences that start with $0.x_1x_2\ldots x_n$

Arithmetic coding

- Under the probability transform, this interval gets mapped into another interval, $[F_Y(0.x_1x_2\cdots x_n), F_Y(0.x_1x_2\cdots x_n+(1/2)^n))$, whose length is equal to $P_X(x_1,x_2,\ldots,x_n)$, the sum of the probabilities of all infinite sequences that start with $0.x_1x_2\cdots x_n$
- Under the probability inverse transform, any real number u within this interval maps into a sequence that starts with $x_1x_2\cdots x_n$, and therefore given u and n, we can reconstruct $x_1x_2\cdots x_n$

- The Shannon–Fano–Elias is a prefix-free code of length

$$\lceil \log[1/p(x_1,x_2,\ldots,x_n)] \rceil + 1 \approx \log[1/p(x_1,x_2,\ldots,x_n)] + 2 \text{ bits}$$

Sayood insists on this as the required CW length

Cover, instead, adds one bit more

and, therefore, it is possible to encode the sequence $x_1,x_2,\ldots,x_n$ with this length

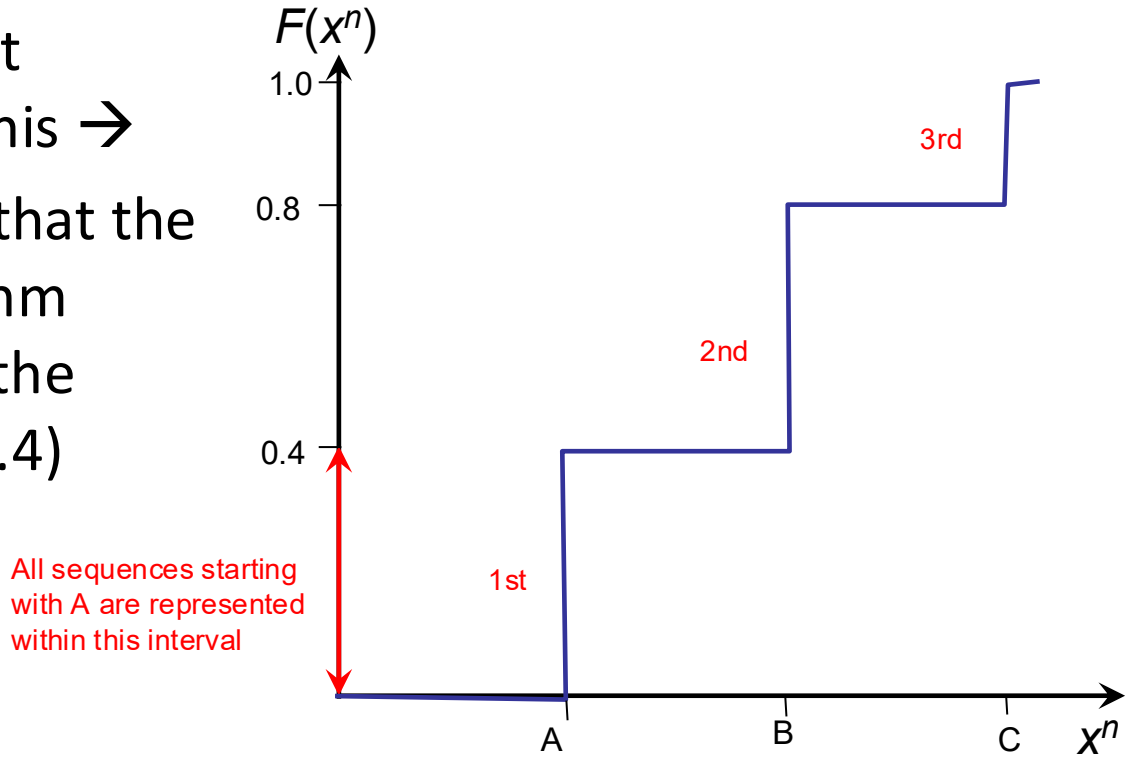
- Note that $\log[1/p(x_1,x_2,\ldots,x_n)]$ is the ideal CW length for x^n

Arithmetic coding

- Encoding the sequence with the CDF described above assumes arbitrary accuracy for the computation
  - In practice, we implement all numbers with finite precision
- The objective of the practical arithmetic coding algorithm is to represent a sequence of RVs by a subinterval in [0,1)
- As the algorithm observes more input symbols, the length of the subinterval corresponding to the input sequence decreases
- As the top end of the interval and the bottom end of the interval get closer, they begin to agree in the first few bits
  - These will be the first bits of the output sequence
- As soon as the two ends agree, we output the corresponding bits
- We can therefore move these bits out of the calculation and scale the remaining intervals so that the entire calculation can be done with finite precision

Example: Arithmetic coding for a 3ary input alphabet

- RV X with ternary alphabet $\mathcal{X}$={A,B,C} and PMF {0.4,0.4,0.2}
- The sequence to encode is "ACAA"
  - Lower limits are $F_l(\cdot)$ = (0,0.4,0.8) and upper limits are $F_h(\cdot)$ = (0.4,0.8,1.0)
- Initially, the input sequence is empty, and the corresponding interval is [0,1)
- The CDF, after the first input symbol, is like this →
- It is easy to calculate that the interval in the algorithm without scaling after the first symbol, A, is [0,0.4)



Example: Arithmetic coding for a 3ary input alphabet

- After the second symbol, C, it becomes [0.32,0.4)

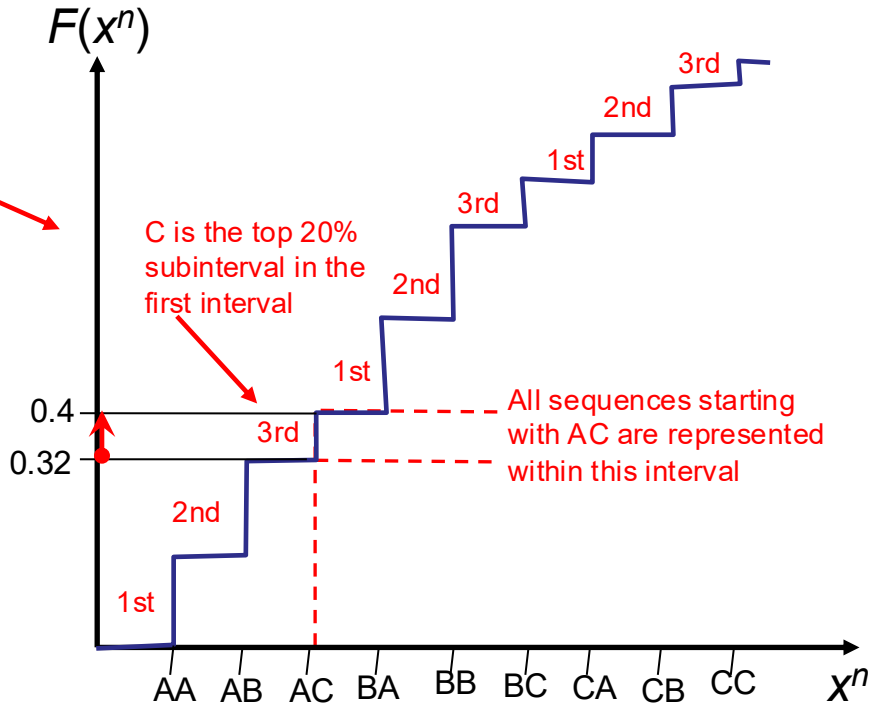
C is in the upper 20% of the parent interval [0.0 0.4), i.e., it starts at the 80% of the parent → 0.4x0.8 = 0.32

- After the third symbol, A, it is [0.32,0.352)

A is in the first 40% of the parent interval [0.32,0.4), i.e., it ends at (0.4-0.32)x0.4 = 0.032 after 0.32

- After the fourth symbol, A, it is [0.32,0.3328)

A is in the first 40% of the parent interval [0.32,0.352), i.e., it ends at (0.352-0.32)x0.4 = 0.0128 after 0.32

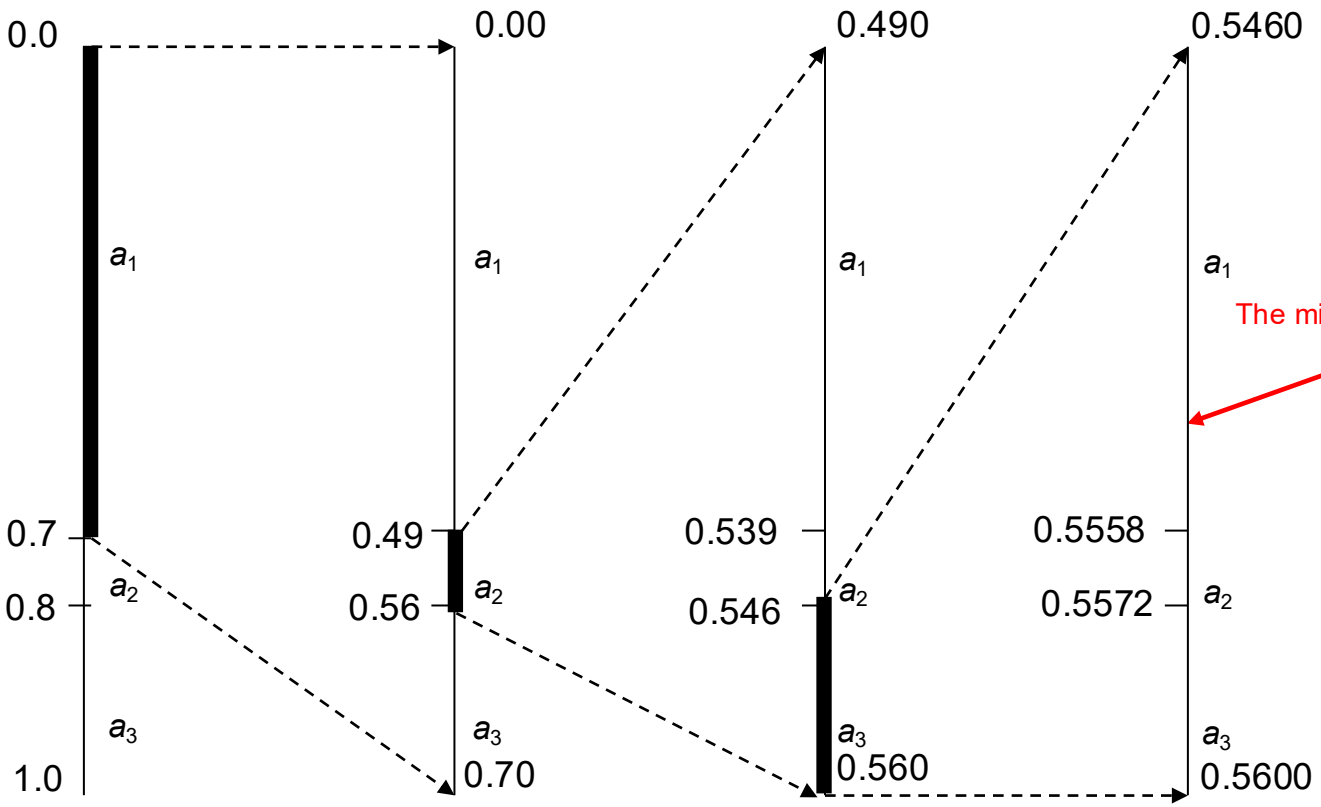


- Since the probability of this sequence is 0.0128, we will use log(1/0.0128) + 1(2) = 7(8).2877 ≈ 8(9) bits to encode the midpoint of the interval sequence using Shannon–Fano–Elias's coding
  - (0.32+0.3328)/2 = 0.3264, which is 0.010100111

0.3264x2=0.6528 → 0
0.6528x2=1.3056 → 1
0.3056x2=0.6112 → 0
0.6112x2=1.2224 → 1
0.2224x2=0.4448 → 0
0.4448x2=0.8896 → 0
0.8896x2=1.7792 → 1
0.7792x2=1.5584 → 1 (Sayood)
0.5584x2=1.1168 → 1 (Cover)
stop

Another example

- Consider a 3-letter alph. $\mathcal{A} = \{a_1, a_2, a_3\}$ and $\mathbf{p} = \{0.7, 0.1, 0.2\}$
  - Then, $F_X(x) = \{0.7, 0.8, 1\}$
- Encode the sequence $a_1 a_2 a_3 \ldots$



Arithmetic coding

- In summary, given any length n and PMF $q(x_1x_2\cdots x_n)$, arithmetic coding encodes the sequence $x_1x_2\cdots x_n$ in a code of length $\log[1/q(x_1x_2\cdots x_n)] + 1(2)$ bits
- If the source is i.i.d. and the assumed distribution q is equal to the true distribution p of the data, this procedure achieves an average length for the block that is within 1(2) bits of the entropy
- Although this is not necessarily optimal for any fixed block length, the procedure is incremental and can be used for any blocklength
  - Huffman codes designed for the PMF could have a lower average CW length

- Arithmetic coding is a winner when the PMF is **highly skewed** (i.e., there is one symbol much more probable than the rest), because it avoids the generation of Huffman tables of large sizes (i.e., need to group symbols to closely approach the entropy)

Lempel-Ziv coding

- We now discuss a popular class of techniques for SC that are simple to implement and universally optimal
  - Their asymptotic compression rate approaches the entropy rate of the source for any stationary ergodic source
- This class of algorithms is termed **Lempel–Ziv**, named after the authors
- The algorithms could also be described as **adaptive dictionary compression** algorithms

- Examples of dictionaries for compression
  - Telegraph era: cost depended on the number of letters, and large companies used CWs from a codebook for the frequently used phrases
  - Greetings telegrams (not the app!) in modern India (till 2013): e.g., “25:Merry Christmas”, a person sends a greeting specifying a number, which is used to generate the actual greeting at the destination

Antique dictionary compressors

A A L 711 . . . Vessel has *anchored*.
A A M 712 . . . Let go another *anchor* and veer out plenty of chain.
A A N 713 . . . We cannot weigh our *anchor*.
A A O 714 . . . We have lost an *anchor*.
A A P 715 . . . We have lost all our *anchors* and must stand out to sea.
A A Q 716 . . . We have lost all our *anchors* and must go ashore.
A A R 717 . . . The following vessel has lost her *anchors*.
A A S 718 . . . Send an *anchor* to us immediately, or as soon as the weather will permit.

NOTE.—Weight can be *signalled* after, in cwts.

A A T 719 . . . Send an *anchor* and *chain* to us immediately, or as soon as the weather will permit.

NOTE.—Size of chain to be *signalled* in quarter inches, (example,) if a chain an inch and a half is required, shew the numbers which represent the number six, which would be *six quarter* inches, or one and a half inches.

A A U 720 . . . Send an *anchor* immediately, or as soon as the weather will permit, to the following vessel, weight in cwts.
A A V 721 . . . Send an *anchor* and chain immediately, or as soon as weather will permit, to the following vessel.
A A W 722 . . . Prepare to *anchor*.
A A X 723 . . . *Anchor* immediately.
A A Y 724 . . . Do not *anchor*.
A A Z 725 . . . Will you *anchor*?
A B A 726 . . . I shall *anchor*.
A B B 727 . . . I do not intend to *anchor*.
A B C 728 ANCHORAGE.
A B D 729 . . . *Anchorage* is bad.
A B E 730 . . . *Anchorage* is good.
A B F 731 . . . *Anchorage* is not safe, if the wind blows from.

STANDARD PHRASES FOR GREETING TELEGRAMS.

1. HEARTIEST DIWALI GREETINGS.
2. ID MUBARAK.
3. HEARTIEST BIJOYA GREETINGS.
4. A HAPPY NEW YEAR TO YOU.
5. MANY HAPPY RETURNS OF THE DAY.
6. HEARTIEST CONGRATULATIONS ON THE NEW ARRIVAL.
7. CONGRATULATION ON THE DISTINCTION CONFERRED ON YOU.
8. BEST WISHES FOR LONG AND HAPPY MARRIED LIFE.
9. A MERRY CHRISTMAS TO YOU.
10. HEARTY CONGRATULATIONS ON YOUR SUCCESS IN THE EXAMINATION.
11. BEST WISHES FOR A SAFE AND PLEASANT JOURNEY.
12. HEARTY CONGRATULATIONS ON YOUR SUCCESS IN ELECTION.
13. MANY THANKS FOR YOUR GOOD WISHES WHICH I / WE RECIPROCATE MOST HEARTLY.
14. CONGRATULATIONS.
15. LOVING GREETINGS.
16. MAY HEAVENS CHOICEST BLESSINGS BE SHOWERED ON THE YOUNG COUPLE.
17. WISH YOU BOTH A HAPPY AND PROSPEROUS WEDDED LIFE.
18. KIND REMEMBERANCE AND ALL GOOD WISHES FOR THE INDEPENDENCE DAY.
19. SINCERE GREETINGS FOR THE REPUBLIC DAY, LONG LIVE THE REPUBLIC.
20. HEARTIEST HOLI GREETINGS.
21. WISHING THE FUNCTION EVERY SUCCESS.
22. MANY THANKS FOR YOUR KIND MESSAGE FOR GREETINGS.
23. BEST WISHES FOR YOUR SUCCESS IN THE EXAMINATION.
24. BEST WISHES FOR YOUR SUCCESS IN THE ELECTIONS.
25. BLESSINGS TO THE NEWLY MARRIED COUPLE.
26. HEARTIEST PONGAL GREETINGS.
27. HEARTIEST GURPURB GREETINGS.
28. GREETINGS ON THE OCCASIONS OF 'PARYUSHAN' A DAY OF UNIVERSAL FORGIVENESS.
29. HEARTIEST ONAM GREETINGS.
30. BEST WISHES ON YOUR WEDDING ANNIVERSARY.
31. WISH YOU A HAPPY RETIRED LIFE.
32. WISH YOU A SPEEDY RECOVERY.
33. HEARTIEST UGADI GREETINGS.
34. CONGRATULATIONS ON YOUR VICTORY.
35. WISH YOU A HAPPY [illegible].
36. A HAPPY EASTER.
37. HEARTIEST GREETINGS ON BUDDHA JAYANTI.

gettyimages Hindustan Times

Lempel-Ziv coding

- The idea of adaptive dictionary-based schemes was not explored until Ziv and Lempel wrote their papers in 1977 and 1978
- The two papers describe two distinct versions of the algorithm
  - **LZ77** (sliding window Lempel–Ziv) or LZ1
  - **LZ78** (tree-structured Lempel–Ziv) or LZ2

- The key idea of the Lempel–Ziv algorithm is to parse the string to compress into phrases and to replace phrases by pointers to where the same string has occurred in the past
- The differences between the algorithms is based on differences in the set of possible match locations (and match lengths) the algorithm allows

Sliding Window Lempel–Ziv Algorithm (LZ77)

- The algorithm in the 1977 paper encodes a string by finding the longest match anywhere within a window of past symbols and represents the string by a pointer to the match location (within the window) and the match length
- Many variations, we describe the Storer-Szymanski flavor
- Assume a string $x_1x_2\cdots$ from a finite alphabet
- A **parsing** S divides $x_1x_2\cdots x_n$ into comma-separated phrases
  - Let W be the length of the window
- Assume that we have compressed the string until time $i-1$
- To find the next phrase, find the largest k such that for some j, $i-1-W \le j \le i-1$, the string of length k starting at x_j is equal to the string (of length k) starting at x_i (i.e., $x_{j+l} = x_{i+l}$ for all $0 \le l < k$)

Sliding Window Lempel–Ziv Algorithm (LZ77)

- The next phrase is then of length k (i.e., $x_i \cdots x_{i+k-1}$) and is represented by the pointer-length (offset) pair (P,L), where P is the location of the beginning of the match and L is its length
- If a match is not found in the window, the next character is sent uncompressed
- To distinguish between these two cases, a flag bit is needed, and hence the phrases are of two types: (F,P,L) or (F,C), where C represents an uncompressed character
- Note that the target of a (P,L) pair could extend beyond the window W, so that it overlaps with the new phrase
  - In theory, this match could be arbitrarily long
  - In practice, the maximum phrase length is restricted

Example of LZ77

- Let $W = 4$, the string "ABBABBABBBAABABA" (16 symbols), and an initial empty window. The string will be parsed as

  A, B, B, ABBABB, BA, A, BA, BA

- It is then represented by the sequence of 8 "pointers"

  (0,A) (0,B) (1,1,1) (1,3,6) (1,4,2) (1,1,1) (1,3,2) (1,2,2)

  No match; No match; Match, go back 1, length 1; Match, go back 3, length 6 (repeat); Match, go back 4, length 2; Match, go back 1, length 1 (inefficient); Match, go back 3, length 2; Match, go back 2, length 2

  - The bitstream (40 bits) is

    00 01 100000 110101 111001 100000 110001 101001

    Note that P=0 or L=0 are not possible → save bits using (F,P-1,L-1)

- The **flag bit** ***F*** = 0/1 denotes match/no match inside *W*: match ***location P*** is measured backward from the end of the window; match **length *L*** may include extension beyond *W*'s end
- Here we have represented every match using the (*F*,*P*,*L*) pair
  - However, it might be more efficient to represent short matches as uncompressed characters, e.g., (0,A) instead of (1,1,1)

Sliding Window Lempel–Ziv Algorithm (LZ77)

- We can view this algorithm as using a **dictionary** that consists of all substrings of the string in the window *W* and of all single characters
- The algorithm finds the longest match within the dictionary and sends a pointer to that match

- This simple variation of LZ77 is asymptotically optimal

- Most practical implementations of LZ77, such as gzip and pkzip, are also based on this version of LZ77
- gzip and PNG use LZ77 with variable length coding

Tree-Structured Lempel–Ziv algorithms

- In the 1978 paper, the algorithm parses a string into phrases, where each phrase is the shortest phrase not seen earlier
- Can be viewed as building a dictionary in the form of a tree, where the nodes correspond to seen phrases
- The algorithm is simple to implement and popular for file compression on computers because of its speed and efficiency
  - It was also used for data compression in “high-speed” modems
- The source sequence is sequentially parsed into strings that have not appeared so far

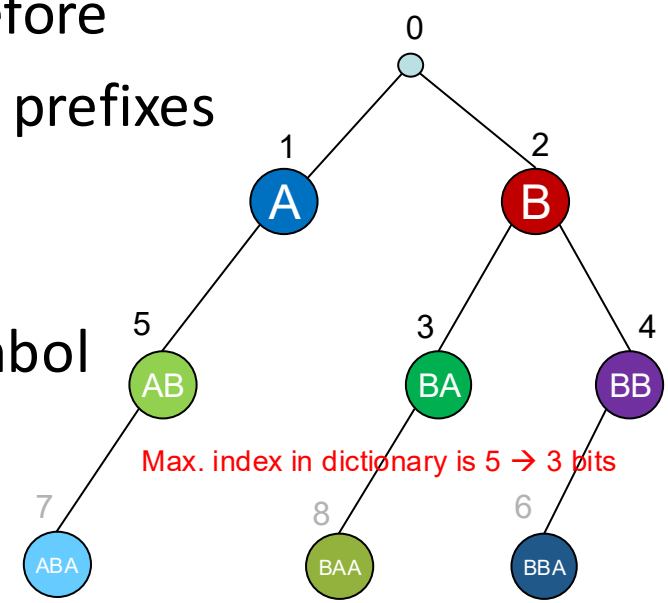
Example of LZ78

- The same string "ABBABBABBBAABABAA" this time is parsed as

  A, B, BA, BB, AB, BBA, ABA, BAA

- After every comma, we look along the input sequence until we come to the shortest string not marked before
- Since this is the shortest such string, all its prefixes must have occurred earlier
  - Thus, we can build up a tree of these phrases
- The string consisting of all but the last symbol of this string must have occurred earlier
- Encode the phrase with the prefix location and the last symbol value
- The representation and bitstream (32 bits) are

new	new	occurred	occurred	occurred	occurred	occurred	occurred
(0,A),	(0,B),	(2,A),	(2,B),	(1,B),	(4,A),	(5,A),	(3,A)
0000	0001	0100	0101	0011	1000	1010	0110



Can we save more bits?

Examples of LZ77/LZ78

- Sending an uncompressed character in each phrase results in a loss of efficiency
- In a first pass, we parse the string and calculate $c(n)$ substrings
  - We allot $\log[c(n)]$ bits to the pointers in the algorithm
- In a second pass, we calculate the pointers and produce the coded string
- The algorithm can be modified so that it requires only one pass over the string and uses fewer bits for the initial pointers
- This variation, due to Welch (LZW), is the basis of most practical implementations of LZ78, such as *compress* on Unix, in compression in modems, and in the GIF image file format

Example of LZW

- Prepopulate the dictionary and insert new patterns every time
- Encode the pattern already present in the dictionary

Index	Entry
1	A
2	B

(1)

Index	Entry
1	A
2	B
3	AB

(2)

Index	Entry
1	A
2	B
3	AB
4	BB

(2)

Index	Entry
1	A
2	B
3	AB
4	BB
5	BA

(3)

Index	Entry
1	A
2	B
3	AB
4	BB
5	BA
6	ABB

(5)

Index	Entry
1	A
2	B
3	AB
4	BB
5	BA
6	ABB
7	BAB

(4)

Index	Entry
1	A
2	B
3	AB
4	BB
5	BA
6	ABB
7	BAB
8	BBB

(5)

Index	Entry
1	A
2	B
3	AB
…	…
8	BBB
9	BAA

(3)

Index	Entry
1	A
2	B
3	AB
…	…
8	BBB
9	BAA
10	ABA

(10)

Index	Entry
1	A
2	B
3	AB
…	…
8	BBB
9	BAA
10	ABA
11	ABAA

(1)

Index	Entry
1	A
2	B
3	AB
…	…
8	BBB
9	BAA
10	ABA
11	ABAA

A,B,B,AB,BA,BB,BA,AB,ABA,A

0,01,01,010,100,011,100,0010,
1001,0000

29 bits

Optimality of LZ77

- The authors proved that LZ77 compressed any string as well as any finite-state compressor acting on that string
- However, they did not prove that LZ77 achieved asymptotic optimality
  - I.e., that the compression ratio converged to the entropy for an ergodic source
- This result was proved by Wyner, Ziv, and Kac
  - The average length of time that you need to wait to see a particular symbol is the reciprocal of the probability of a symbol
- Thus, we are likely to see the high-probability strings within the window and encode these strings efficiently
- The strings that we do not find within the window have low probability, so that, asymptotically, they do not influence the compression achieved

Recurrence time coding for LZ77

- Let $R_n(X^n)$ be the RV describing the last time in the past that we have seen a block of n symbols X^n
- Then, the cost of encoding is the cost of representing the outcomes r_n, which for large n gives

$$(1/n)\ \mathrm{E}\{\log R_n\} \rightarrow H(\mathcal{X})$$

- Thus, encoding by describing the recurrence time is asymptotically optimal

Optimality of LZ78

- The proof of the optimality of LZ78 is different from that for LZ77
- In LZ78 the input sequence is parsed into phrases, each phrase being the shortest string that has not been seen so far
- If $l(x^n)$ is the description length of the parsed sequence, then

$$\limsup (1/n)\, l(X^n) \le H(\mathcal{X})$$

with probability 1 for every stationary ergodic SP $\{X_i\}$

- The Lempel–Ziv codes are simple examples of a universal code (i.e., a code that does not depend on the pdf of the source)
- This code can be used without knowledge of the source pdf and yet will achieve an asymptotic compression equal to the entropy rate of the source